

## HRTIM with divergent channel frequencies

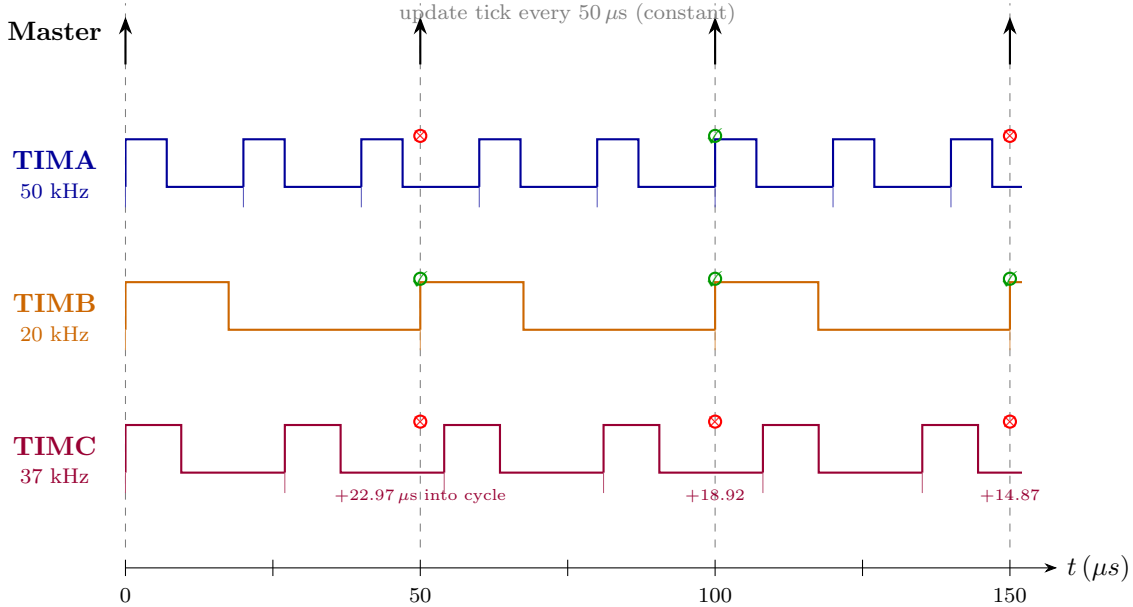
What does the Master (update-tick) period look like?

TIMA = 50 kHz (20.000  $\mu\text{s}$ )    TIMB = 20 kHz (50.000  $\mu\text{s}$ )    TIMC = 37 kHz (27.027  $\mu\text{s}$ )    all updates transfer at Master period start

**The core mechanic.** With preload enabled, a PFM-table write to a channel’s PER/CMP sits in the shadow register until the Master update tick. The transfer itself is always clean — the question is *where in the channel’s own cycle* the tick lands. If the tick coincides with the channel’s own rollover (✓), the new period simply defines the next cycle. If it lands mid-cycle (✗), the active PER changes under a running counter: the cycle in progress is completed against the *new* PER, producing one distorted cycle — and if the counter has already passed the new (smaller) PER value, the compare is missed and the counter runs long (up to a 16-bit wrap), i.e. one stretched or runt pulse. So the Master period should be chosen to put ticks on channel rollovers — and with 50/20/37 kHz no single small constant period can do that for all three channels at once.

### Possibility 1 — constant Master period

Take the natural “central” choice  $T_M = 50 \mu\text{s}$  (the slowest channel). B aligns with every tick by construction ( $T_M = T_B$ ); A aligns only at even ticks ( $100 \mu\text{s} = 5T_A$ ); C *never* aligns, because 27.027  $\mu\text{s}$  is incommensurate with 50  $\mu\text{s}$ .



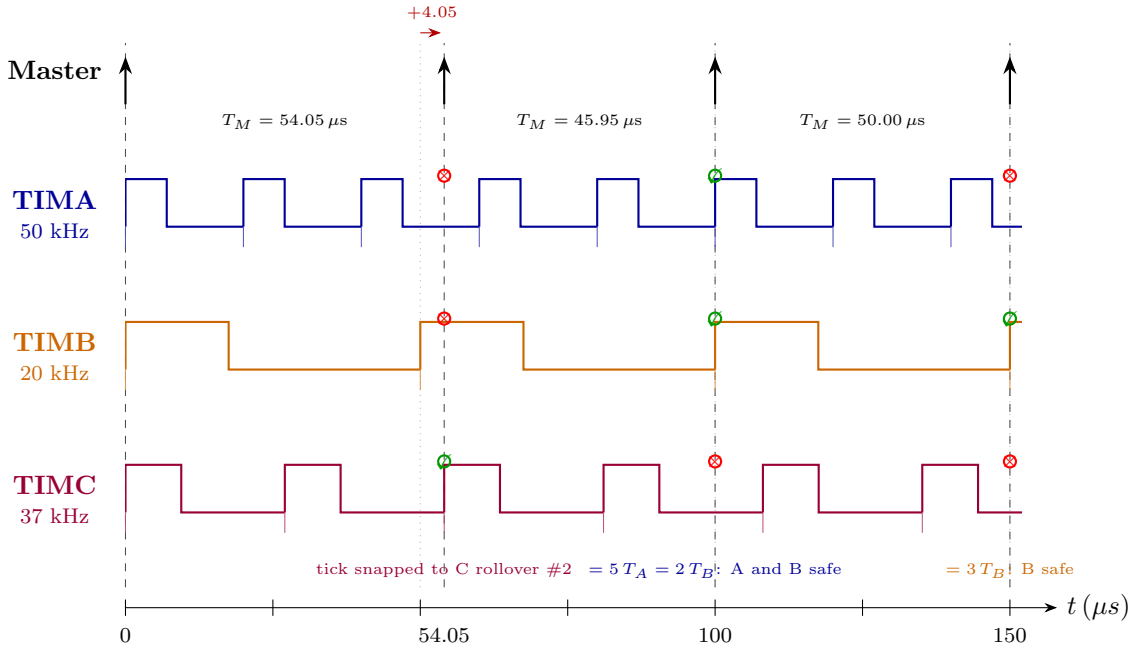
**Reading it:** every tick is safe for B; ticks at odd multiples of 50  $\mu\text{s}$  land 10  $\mu\text{s}$  (half a cycle) into A; ticks land at a *different, drifting* point of C’s cycle each time ( $22.97 \rightarrow 18.92 \rightarrow 14.87 \mu\text{s} \dots$ ), because the offset advances by  $50 \bmod 27.027 = 22.97 \mu\text{s}$  per tick. A constant central Master period therefore forces one of:

- *accept mid-cycle transfers* on unaligned channels (one distorted cycle per PER/CMP change on that channel — harmless if the table rarely changes that channel, unacceptable during fast ramps);
- *only change a channel’s values on its aligned ticks* (A updatable every 100  $\mu\text{s}$ , B every 50  $\mu\text{s}$ , C never — broken for C);

- *make the constant period a true common multiple*: any set of integer-kHz frequencies aligns at  $T_M = 1\text{ ms}$  (here  $= 50 T_A = 20 T_B = 37 T_C$ ). Every tick is then safe for every channel, but the table can only step once per millisecond, and  $1\text{ ms} = 170,000\text{ counts}$  exceeds the 16-bit Master PER at  $\div 1$  prescale — it needs the Master prescaler ( $\div 4 \Rightarrow 42,500\text{ counts}$ ) or the repetition counter.

## Possibility 2 — Master period varies slightly around a central value

Let the Master period hover near  $50\text{ }\mu\text{s}$  but re-target *each individual tick* to land exactly on the rollover of whichever channel(s) need new values at that step. The wobble is bounded: the nearest rollover of any channel is never more than half that channel’s period away, so  $T_M$  stays within  $\pm T_{\max}/2$  ( $\pm 25\text{ }\mu\text{s}$  worst case here, usually far less).



**Reading it:** the Master period sequence is  $54.05, 45.95, 50.00\text{ }\mu\text{s}$  — each within  $\pm 8\%$  of the  $50\text{ }\mu\text{s}$  center. Tick 1 was stretched to C’s rollover so C’s new PER/CMP transfer cleanly; tick 2 was shortened back to  $100\text{ }\mu\text{s}$ , which happens to be a rollover of both A and B; tick 3 serves B. The crosses do not disappear — with mutually incommensurate channels *no single tick can be safe for all three* — but now the firmware *chooses* which channel each tick is safe for, so every channel’s updates can be scheduled onto its own safe ticks.

**Costs of possibility 2:** the Master PER must be recomputed and rewritten every tick (ISR does modular arithmetic: next boundary = nearest rollover of the target channel to the central grid point); “dwell” is no longer a fixed wall-clock quantum (ticks vary a few percent); and if two channels need updating on the same tick, one of them waits for its next safe tick (bounded by one channel period). If the three frequencies are chosen from an integer-kHz grid, possibility 1’s LCM variant ( $T_M = 1\text{ ms}$ , all ticks safe for all channels) is simpler whenever a  $1\text{ ms}$  table-step granularity is acceptable.

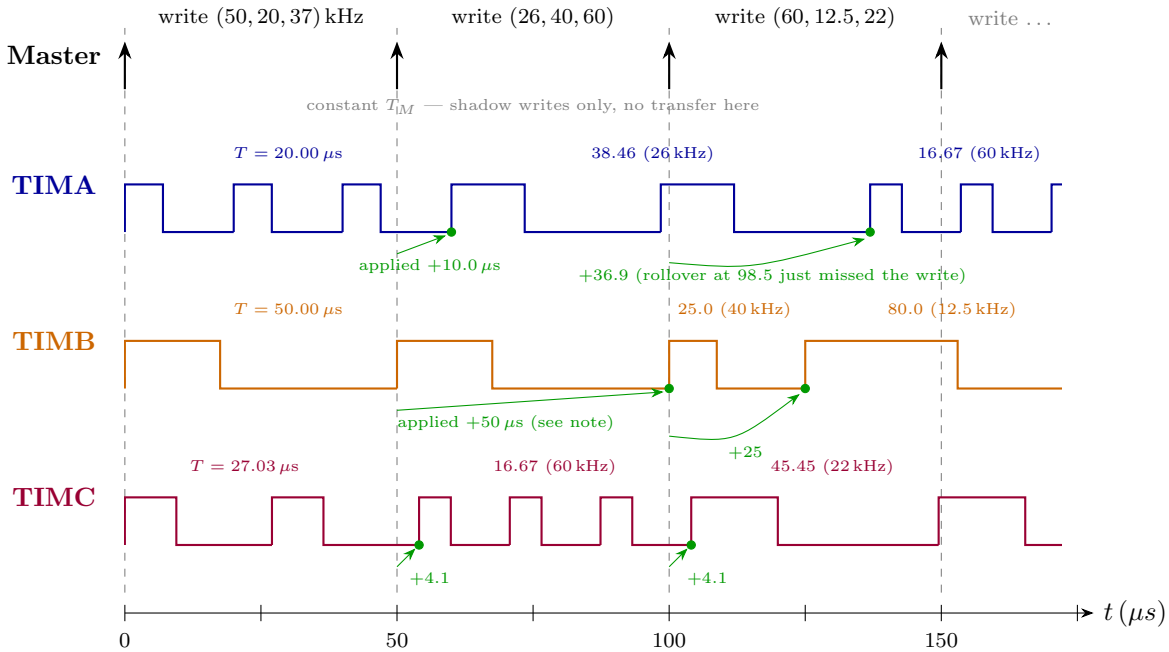
**One more practical note:**  $37\text{ kHz}$  is not exactly representable at  $170\text{ MHz}$  ( $170\text{ MHz}/37\text{ kHz} = 4594.59\text{ counts} \Rightarrow \text{PER} = 4595 \Rightarrow 36.9967\text{ kHz}$ ). Any alignment scheme that assumes exact integer-kHz ratios (including the  $1\text{ ms}$  LCM) inherits a slow phase creep of  $\sim 90\text{ ppm}$  on that channel unless the PER value is dithered between 4594 and 4595.

### Possibility 3 — every step changes every channel: per-channel update-on-own-rollover

Now suppose the table changes *all three* periods on *every* Master tick — step  $n$ : (50, 20, 37) kHz, step  $n+1$ : (26, 40, 60) kHz, step  $n+2$ : (60, 12.5, 22) kHz, ... — no relationship between channels. (These jumps are deliberately exaggerated, far beyond any realistic slew limit, so the shadow→active mechanics are visible; a slew-limited table behaves identically with smaller steps.) Neither previous scheme copes: possibility 1 has no safe tick for an unaligned channel, and possibility 2 can only make each tick safe for *one* channel, yet all three now need new values every tick.

**The fix is to stop transferring at the Master tick altogether.** HRTIM allows each timer’s shadow→active transfer to be triggered by *its own* counter roll-over (per-timer update-on-reset/roll-over, `ResetUpdate = HRTIM_TIMUPDATEONRESET_ENABLED` with `UpdateTrigger = NONE` and `ResetTrigger = NONE`; RM0440’s TxRSTU covers roll-over at PER, not just external resets). The roles then separate cleanly:

- **Master tick** (constant period — no wobble needed): the table metronome. The ISR steps the table and *writes* the three new PER/CMP pairs into the shadow registers. Writing shadows is always safe, at any time.
- **Each channel’s own roll-over**: the hardware *applies* whatever is in that channel’s shadow at that instant. By construction the transfer lands exactly on a cycle boundary — every cycle, on every channel, is a “green-check tick” for itself. The mid-cycle hazard is gone by design, so the slew limit is no longer needed for hardware safety at all (it only shapes the trajectory).



Dashed lines are Master ticks (constant 50  $\mu\text{s}$ ): the ISR *writes* the next  $(f_A, f_B, f_C)$  tuple there. Green dots are where each channel’s hardware *applies* its new period — always at its own next roll-over, never mid-cycle. Every transition is a clean, complete cycle at one period followed by a clean, complete cycle at the next.

**So: yes, this works**, with no required relationship between the three frequencies and no Master-period gymnastics. The properties to design around:

- **Bounded staleness, not glitches.** A value written at tick  $n$  takes effect up to one *channel* period late (A: +10.0 then +36.9  $\mu\text{s}$  above — the second is nearly a whole period because A’s roll-over at 98.5  $\mu\text{s}$  lands just *before* the  $t=100$  write; C: +4.1  $\mu\text{s}$  both times). With a slew-limited table the applied value is at most one step behind the commanded trajectory — a small transport delay, relevant only to control-loop phase margin, not to waveform integrity.
- **Slow channels subsample the table.** A channel whose period is  $\geq T_M$  physically cannot change frequency more often than once per own cycle. TIMB above shows both effects: its  $t=50$  roll-over coincides with the tick, and since the ISR write lands nanoseconds *after* the boundary, the transfer at that instant uses the old shadow — so step  $n+1$  applies a full 50  $\mu\text{s}$  late, and if B’s period exceeded  $T_M$ , intermediate table steps would be skipped entirely (harmless when slew-limited: skipped values lie between their neighbors). Deterministic, one-sided, easy to account for.
- **PER/CMP coherency.** PER and CMP for one channel are two register writes; a roll-over landing between them applies new-PER with old-CMP for one cycle. The slew limit bounds this to a sub-percent duty error for one cycle. If even that matters, HRTIM’s update-gating (UPDGAT) or a DMA-burst transfer makes the pair atomic.
- **What is given up:** the Master-anchored  $120^\circ/240^\circ$  interleave (channels free-run; inter-channel phase is unconstrained and drifts), and the fixed carrier-period dwell quantum ( $T_M$  is now an arbitrary constant metronome — pick it as the table’s time resolution, e.g. the 50  $\mu\text{s}$  used here).
- **Firmware delta from today’s code (hrtim.c):** per A/B/C set `ResetTrigger = NONE`, `UpdateTrigger = NONE`, `ResetUpdate = ENABLED` (today: reset from Master PER/CMP1/CMP2, update from Master); Master keeps preload + repetition interrupt as the metronome; `HRTIM1_ApplyPfmStep()` writes three PERs instead of one and drops the Master MCMP1/MCMP2 phase-offset writes.

This is the recommended architecture for the “every step, every channel, arbitrary frequencies” requirement: possibility 1 remains right when frequencies sit on a common grid (LCM metronome, everything stays phase-coherent), possibility 2 when only occasional per-channel retunes are needed, and possibility 3 when the table is a continuous three-channel frequency trajectory.